

Introduction

A transportation network is a system in which cities are represented as nodes and the roads connecting them are represented as edges. Analyzing such networks is an important application of graph theory, widely used in real-world scenarios like navigation systems, logistics planning, and network routing. This project focuses on designing and implementing an Interactive Transportation Network Application using Java. The application allows users to create a network of cities and roads and apply fundamental graph algorithms to analyze it. Algorithms such as Dijkstra’s Algorithm, Breadth-First Search (BFS), and Prim’s Minimum Spanning Tree (MST) are used to solve problems like finding the shortest path, traversing the network, checking connectivity, and minimizing overall cost. The project helps in understanding how theoretical concepts of data structures and algorithms can be applied to practical, real-life problems efficiently.

Objective

Design and implement an interactive application that models a transportation network using cities and roads. The application allows users to construct, visualize, and analyze the network using efficient algorithms for traversal, connectivity, and cost optimization.

Input

- A predefined set of cities is created in the program.
- Roads between cities are defined along with their respective distances (in kilometers).
- A source city is selected from which all network analysis is performed.

Output

- **Shortest Path:** Displays the minimum distance and path from the source city to all other cities using Dijkstra’s Algorithm.
- **Traversal Order:** Shows the order in which cities are visited using Breadth-First Search (BFS).
- **Reachability:** Indicates whether each city is reachable from the source and determines if the network is connected or disconnected.
- **Minimum Cost Network:** Displays the minimum set of roads required to connect all cities along with the total cost using Prim’s Algorithm.

Algorithms Used

Algorithm	Purpose	Data Structure	Complexity
Dijkstra's	Shortest path from source to all cities	Array + visited[]	$O(V^2)$
BFS	City traversal level by level	Queue	$O(V+E)$
BFS (Reachability)	Check if all cities are reachable	Queue + visited[]	$O(V+E)$

Prim's MST	Min roads to connect all cities	Key[] + inMST[]	$O(V^2)$
------------	---------------------------------	-----------------	----------

Assumptions

- The transportation network is represented as an **undirected graph**, meaning roads are two-way.
- All distances between cities are **positive values**.
- A minimum of **5 cities** is required to perform meaningful analysis.
- Each city has a **unique name** (no duplicates allowed).
- The network is **static**, meaning cities and roads do not change during algorithm execution.
- There are **no self-loops** (a city cannot connect to itself).

Sample Output



Transportation Network

=== SHORTEST PATHS ===

Delhi : 0 km

Mumbai : 1400 km

Kolkata : 900 km

Chennai : 2130 km

Hyderabad : 1500 km

Bangalore : 2070 km

=== BFS TRAVERSAL ===

Delhi

Mumbai

Kolkata

Hyderabad

Bangalore

Chennai

=== REACHABILITY ===

Delhi : YES

Mumbai : YES

Kolkata : YES

Chennai : YES

Hyderabad : YES

Bangalore : YES

CONNECTED

=== MIN COST NETWORK ===

Delhi - Mumbai : 1400 km

Delhi - Kolkata : 900 km

Bangalore - Chennai : 350 km

Mumbai - Hyderabad : 700 km

Hyderabad - Bangalore : 570 km

Total Cost = 3920 km

Output Explanation

1. Shortest Paths (Dijkstra's Algorithm)

This section displays the minimum distance from the source city (Delhi) to all other cities.

- Delhi has distance **0 km** as it is the source.
- Mumbai (1400 km) and Kolkata (900 km) are directly connected to Delhi.
- Chennai (2130 km) is reached through Hyderabad, giving the shortest route.
- Hyderabad (1500 km) is directly connected.
- Bangalore (2070 km) is reached via Hyderabad.

2. BFS Traversal (Breadth-First Search)

This section shows the order in which cities are visited starting from Delhi.

Traversal order:

- Delhi → Mumbai → Kolkata → Hyderabad → Bangalore → Chennai

3. Reachability Analysis

This checks whether all cities can be reached from the source city.

- All cities are marked **YES**, meaning every city is reachable.
- Therefore, the network is **CONNECTED**.

4. Minimum Cost Network (Prim's MST Algorithm)

This section shows the minimum set of roads required to connect all cities with the least total cost.

Selected roads:

- Delhi – Mumbai (1400 km)
- Delhi – Kolkata (900 km)
- Mumbai – Hyderabad (700 km)
- Hyderabad – Bangalore (570 km)
- Bangalore – Chennai (350 km)

Total Cost = **3920 km**

The output verifies that all algorithms are correctly implemented and successfully analyze the transportation network in terms of shortest path, traversal, connectivity, and cost optimization..

Conclusion

The Interactive Transportation Network Application successfully demonstrates the practical implementation of important graph algorithms using Java. Through this project, a network of cities and connecting roads was created and analyzed efficiently. Algorithms such as Dijkstra's Algorithm, Breadth-First Search (BFS), and Prim's Minimum Spanning Tree were used to solve real-world problems like shortest path finding, network traversal, connectivity checking, and cost optimization. The project highlights how data structures like arrays and queues can be effectively used to represent and process graphs. It also shows the importance of algorithmic efficiency in handling network-related problems. The application provides meaningful outputs such as shortest routes, traversal order, reachability status, and minimum cost network, making it both informative and practical. Overall, this project helped in understanding the real-life applications of graph theory and improved problem-solving and programming skills. It can be further enhanced by adding more advanced features such as dynamic inputs, weighted visualization, and improved user interface.